

Building Nginx Web Server in AWS cloud platform using Terraform

I built a modular, multi-file Terraform configuration to dynamically provision an Amazon Linux 2023 EC2 instance and bootstrap a Nginx web server on AWS.

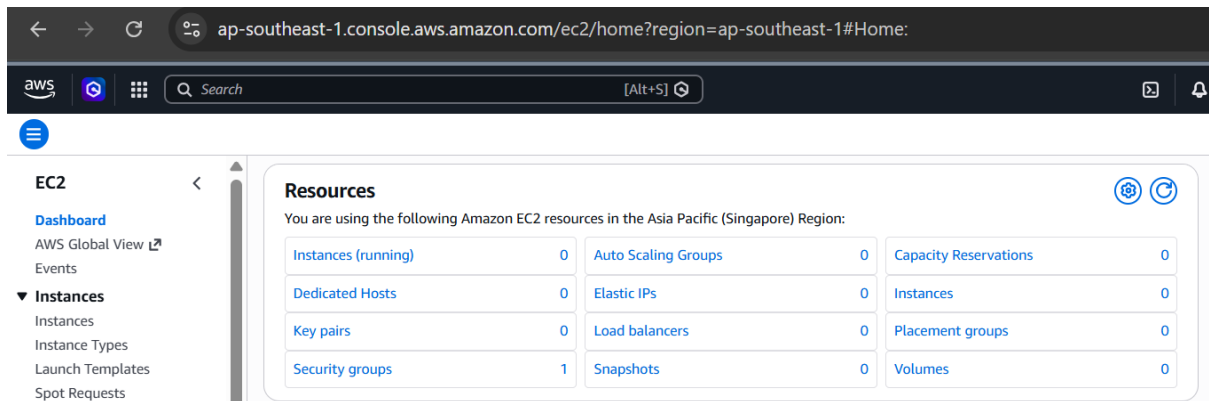
Instead of writing a single bloated `main.tf` file, I focused on infrastructure-as-code (IaC) best practices by decoupling the logic into:

- 📁 providers.tf – Managed AWS provider versions.
- 📁 variables.tf – Parametrizing regions and instance sizing.
- 📁 data-linux.tf – Querying the AWS marketplace for the latest verified AMIs.
- 📁 main.tf – Handling core resource provisioning and security groups.
- 📁 outputs.tf – Delivering the final runtime public endpoints.

💡 Key learnings & troubleshooting along the way:

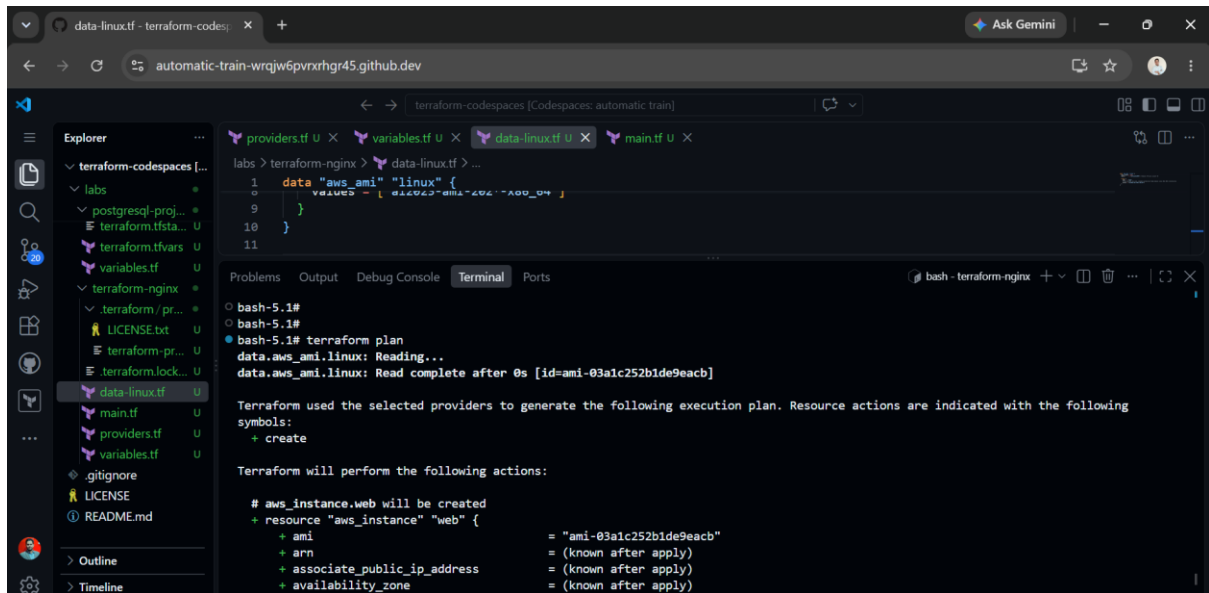
- **Dynamic AMI Filtering:** Learned how to explicitly isolate official Amazon-owned AMIs (using Owner ID 137112412989) while parsing wildcards to match standard production-ready AL2023 architectures.
- **Regional Free Tier Variances:** Found out the hard way that 't2.micro' isn't universally free! Debugged regional variations in ap-southeast-1 (Singapore) and dynamically refactored variables to use 't3.micro' to align with regional free-tier eligibility.
- **Strict CIDR validation:** Fixed syntax bugs by ensuring netmask suffixes (/0) were properly formatted for robust security group ingress/egress rules.

- Before creating Web application instance, there was no instance running on my aws account.



- Once codes are ready then Run terraform validate command to check the execution plan what's actually terraform is going to create / destroy in aws platrom.

Building Nginx Web Server in AWS cloud platform using Terraform



- Before applying, I run terraform state list to check is there any resources are available or not available:

```
bash-5.1# terraform state list
No state file was found!

State management commands require a state file. Run this command
in a directory where Terraform has been run or use the -state flag
to point the command to a specific state location.

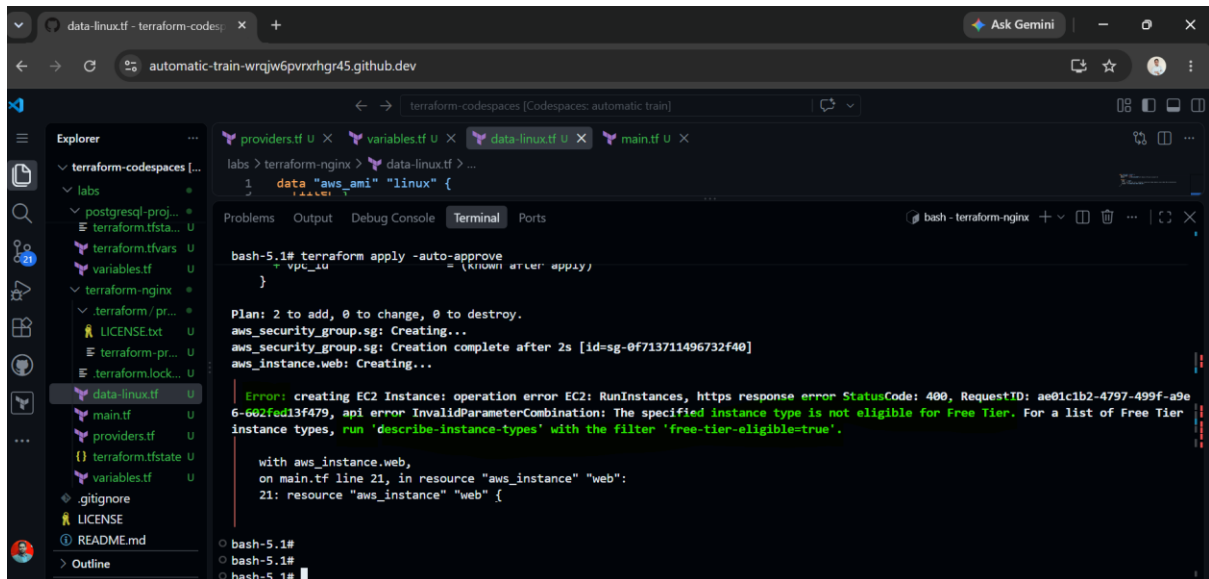
bash-5.1#
```

Output shows no state file was found, means no resources are available currently in our infrastructure.

After applying terraform apply commands we have got error: **instance type is not eligible for free** : to check free tier instance we can run >

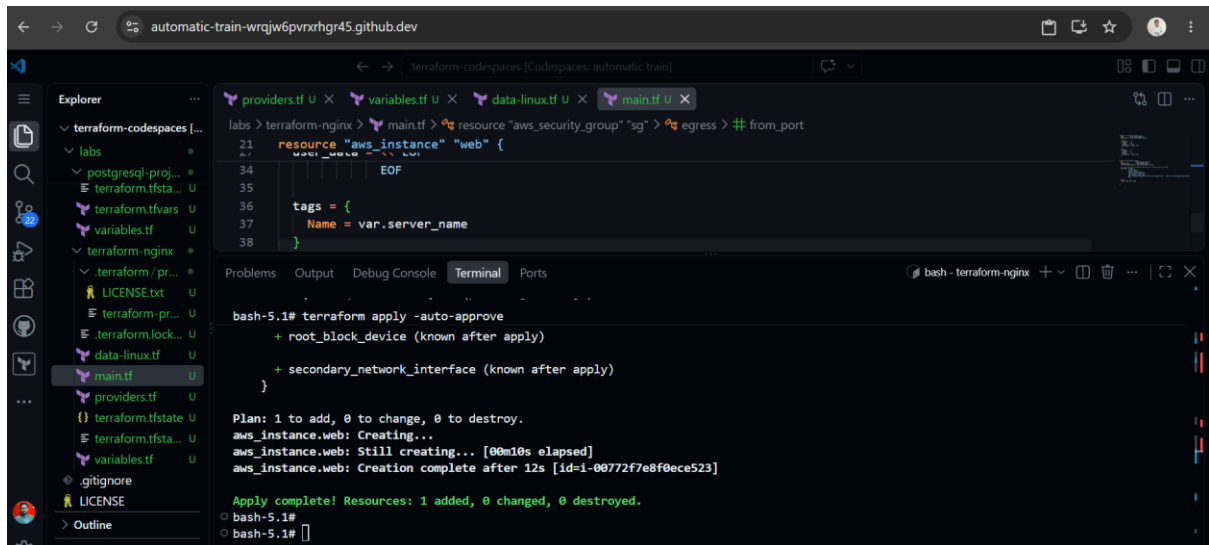
To fix this error, I changed the instance_type value on or around **line 21 of your main.tf** file to a standard Free Tier type, such as t2.micro

Building Nginx Web Server in AWS cloud platform using Terraform



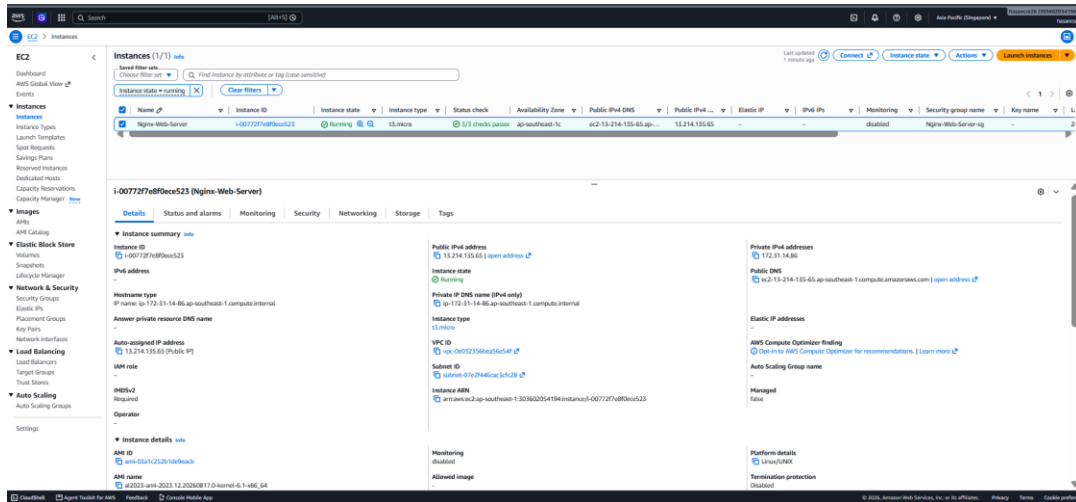
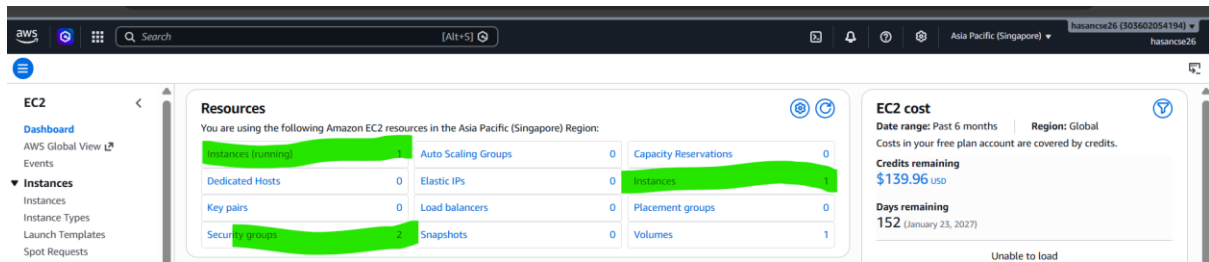
After changing some small fixes in data-linux.tf

```
data "aws_ami" "linux" {
  most_recent = true
  owners      = ["137112412989"] # Amazon official account ID
  filter {
    name = "name"
    # This matches the core, standard production-ready Amazon Linux 2023 AMI architectures
    values = ["al2023-ami-202*-x86_64"]
  }
}
```

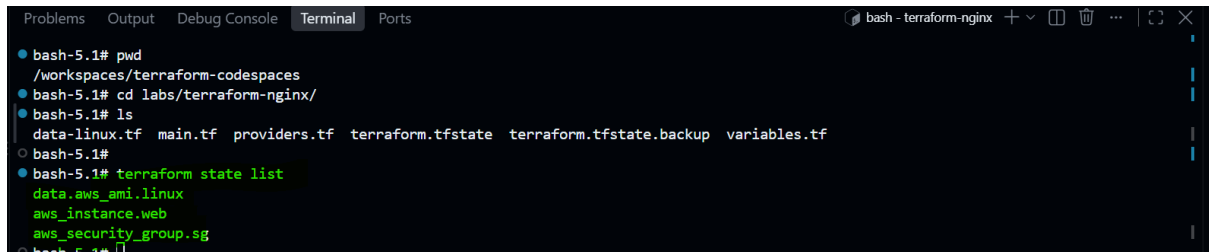


After running terraform apply now we can see there is one instance **nginx-web-server** is running

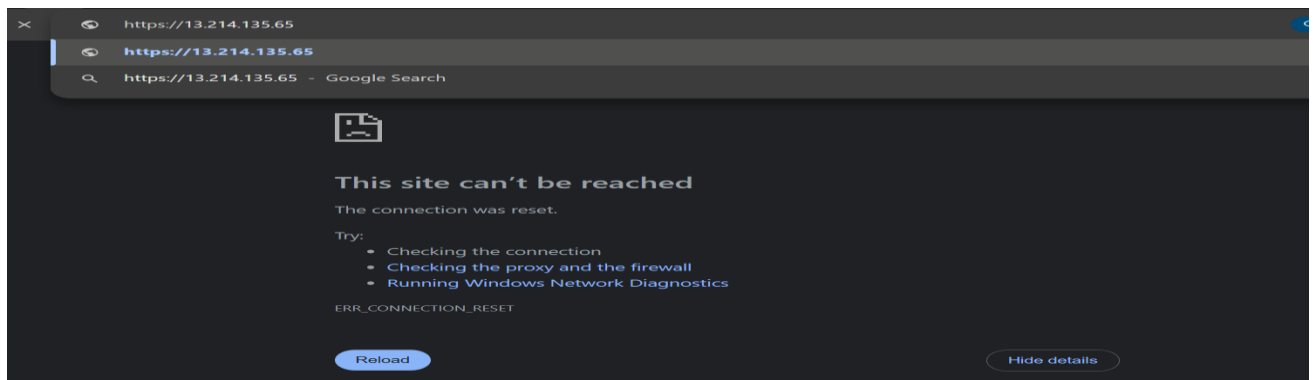
Building Nginx Web Server in AWS cloud platform using Terraform



We can also check the terraform state file:



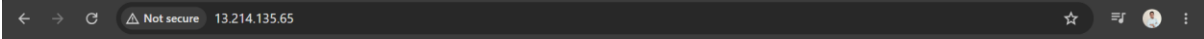
We faced issue while opening this web server The reason website is not loading is because our browser is trying to connect using secure **https://** (Port 443), but our infrastructure configuration is only listening for unencrypted **http://** traffic on Port 80.



Fix

Change the URL in our browser's address bar from **https://13.214.135.65** to standard unencrypted HTTP:

Building Nginx Web Server in AWS cloud platform using Terraform



Nginx deployed via multi-file Terraform!

<http://13.214.135.65>



Terraform-Project1-
Nginx-Web-Server-Bu

To destroy these resources we can use below commands in terraform cli:

- Terraform destroy ----- It will destroy all the resources that we created using this configuration files

```
bash-5.1# terraform destroy
data.aws_ami.linux: Reading...
aws_security_group.sg: Refreshing state... [id=sg-0f713711496732f40]
data.aws_ami.linux: Read complete after 0s [id=ami-0c80b07f87a217cb2]
aws_instance.web: Refreshing state... [id=i-00772f7e8f0ece523]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following
symbols:
  - destroy

Terraform will perform the following actions:

# aws_instance.web will be destroyed
- resource "aws_instance" "web" {
  - ami              = "ami-03a1c252b1de9eacb" -> null
  - arn              = "arn:aws:ec2:ap-southeast-1:303602054194:instance/i-00772f7e8f0ece523" -> null
  - associate_public_ip_address = true -> null
}
```